

CONFIDENTIAL — INTERNAL

DRAFT FOR REVIEW

Production Verification Checklist

Privacy Incident — July 2026

Lexy AI Hiring Platform · Generated 2026-07-07

Every claim is backed by a test, migration, or file — or explicitly marked inferred/unverified.

Why this exists. The security review was verified in **development**. The items below are the things that **cannot be reached from dev** and must be confirmed against the live system. For each item: what to run, the result that means **SAFE**, and the result that means **ESCALATE**. Do not mark the incident closed until every item is SAFE or has a tracked exception.

Reference commits (compare against what is actually deployed):

- Privacy seal + rec-push + enumeration: **2026-07-07 00:42 UTC** — “Privacy incident Phase 3: seal rec-push + close platform-pool enumeration.”
- Candidate-database seal proof test + match-only crash fix: **2026-07-06 23:53 UTC**.
- Durable HTTP access logging (http_access_logs): **2026-07-06 21:06 UTC**.

Item 1 — Do the previously-leaky endpoints exist in prod, and does the *sealed* code path run there?

Endpoints in question

- GET /api/candidates (employer candidate list)
- GET /api/tenants/:id/candidate-database (employer shared-pool browse)
- GET /api/candidates/:candidateId/career-recording
- GET /api/candidates/:candidateId/career-profile
- The two recommendation-push paths (runPlatformRecommendationForJob , runPlatformRecommendationScan in platform-recommendation-engine.ts) — these are background engines, not directly curl-able; verify via their output (talent_pool_submissions) in Item 1c.

1a. Endpoints are live. Against the prod base URL, authenticated as a normal **employer tenant user that does NOT have candidate-database access**:

```
curl -sS -o /dev/null -w '%{http_code}\n' \
-H "Authorization: Bearer <prod-employer-token-no-pool-access>" \
https://<PROD_HOST>/api/candidates
```

- **SAFE:** endpoint responds (200) **and** the JSON contains **no** rows with pool == "platform" / no shared-pool candidates. (A licence-less tenant must see only its own rows.)
- **ESCALATE:** any platform-pool candidate appears for a tenant without candidate-database access.

1b. The seal actually bars a *barred* candidate. Pick (or seed) a platform-pool candidate who has a privacy flag set — one of hideFromCurrentEmployer / discoveryPaused / a blockedCompanyDomains entry matching the test tenant / matchOnlyVisibility — or doNotContact / dataErasedAt . As an employer tenant **with** candidate-database access:

```
# list/browse must NOT contain the barred candidate id
curl -sS -H "Authorization: Bearer <prod-employer-token-WITH-pool-access>" \
  "https://<PROD_HOST>/api/tenants/<TENANT_ID>/candidate-database" | \
grep -c "<BARRED_CANDIDATE_ID>"
```

```
# by-id detail surfaces must 404 for the barred candidate
curl -sS -o /dev/null -w '%{http_code}\n' \
  -H "Authorization: Bearer <prod-employer-token-WITH-pool-access>" \
  "https://<PROD_HOST>/api/candidates/<BARRED_CANDIDATE_ID>/career-profile"
```

- **SAFE:** `grep -c` returns `0`, and the career-profile / career-recording by-id calls return **404**. Also confirm a *compliant* control candidate IS returned (so you know the surface isn't just empty).
- **ESCALATE:** the barred candidate appears in the list, or a by-id call returns **200** with the barred candidate's data.

1c. Rec-push does not recommend a barred candidate. In the prod DB, check the recommendation output table for any barred candidate pushed to an employer:

```
-- Any submission whose candidate currently has a privacy/compliance bar set?
SELECT s.id, s.candidate_id, s.client_tenant_id, s.pushed_at
FROM talent_pool_submissions s
JOIN candidates c ON c.id = s.candidate_id
WHERE (c.do_not_contact = true
       OR c.data_erased_at IS NOT NULL
       OR c.discovery_paused = true
       OR c.hide_from_current_employer = true)
ORDER BY s.pushed_at DESC
LIMIT 100;
```

- **SAFE:** zero rows (no barred candidate was pushed to an employer).
- **ESCALATE:** any row — a barred candidate reached an employer via recommendations; capture `client_tenant_id` and `pushed_at` for scope.
- *(Adjust column names to the prod schema; match-only/blocked-domain bars are relational — confirm with the same predicate logic the app's `applyCandidatePrivacyFilter` uses.)*

Item 2 — Do prod access logs show hits on those endpoints against platform-pool rows with privacy flags set?

Query the durable access-log table (Item is bounded by when logging was deployed — see the **honest limit** below):

```
-- NOTE: http_access_logs stores `method` as its OWN column and `route_pattern`
-- as the registered path only (req.baseUrl + req.route.path), e.g. '/api/candidates'
-- with NO method prefix. The tenant route parameter is named `:tenantId`.
SELECT occurred_at, method, route_pattern, status_code, user_id, tenant_id
FROM http_access_logs
WHERE method = 'GET'
      AND route_pattern IN (
        '/api/candidates',
        '/api/tenants/:tenantId/candidate-database',
        '/api/candidates/:candidateId/career-recording',
        '/api/candidates/:candidateId/career-profile'
      )
```

```
ORDER BY occurred_at DESC
LIMIT 500;
```

Confirm the exact `route_pattern` strings against the prod table first (they are the registered patterns, not the raw URLs): `SELECT DISTINCT route_pattern FROM http_access_logs WHERE route_pattern LIKE '%candidate%';`

- **SAFE (as far as it can go):** hits to these routes come only from tenants that legitimately have candidate-database access, and the by-id career routes show **404** for barred candidates.
- **ESCALATE:** by-id career routes returning **200** to a tenant that should not see that candidate; or list/browse hits from licence-less tenants.

HONEST LIMITS of this item — do not overclaim:

- `http_access_logs` records the **registered route pattern only** and **deliberately does not store the raw URL, the `:candidateId`, request bodies, query strings, or the response**. So the log can show *that* an endpoint was hit and by *which tenant/user*, but it **cannot by itself prove which candidate row was returned**. Correlating a specific candidate requires joining other evidence (e.g. Item 1c output tables), not the access log alone.
- The log only goes back to when the `0045_http_access_logs` migration was **deployed to prod** (dev commit 2026-07-06 21:06 UTC; prod deploy date **[UNVERIFIED]** — confirm it). **Anything before that is not queryable**. A clean log is therefore evidence of “no *logged* hits,” **not** proof that no access ever occurred. Record the earliest `occurred_at` in the table as the true start of coverage:

```
SELECT min(occurred_at) AS logging_since FROM http_access_logs;
```

Item 3 — What RLS is ACTUALLY active in prod? (DB-enforced vs app-code-only, by table class)

This is a **platform-wide class distinction**, not a single-table question — see §5 of the Security Review. Two classes matter:

- **Class A** — tables an RLS migration *intended* to protect (`candidates`, `applications`, `interview_sessions`, `jobs`, `sourced_candidates`, `talent_matches`, `resume_screens`, `outreach_*`, `verification_records`, `recruiter_inbox_items`, `pipeline_runs`, `job_pipelines`, ... — the full list is in §5). A correctly-migrated prod **should** have live policies here. This is what Item 3 verifies.
- **Class B** — candidate-data tables that were **never** in any RLS migration (`candidate_job_intelligence`, `hiring_manager_shares`, `interview_summaries`, `candidate_career_profiles`, `candidate_demographics`, `candidate_ai_consent`, `candidate_embeddings`, `candidate_events/outcomes`, `outreach_replies / step_messages`, `ai_decision_log`, ... — full list in §5). **There is nothing to verify for Class B: by design no DB policy exists in any environment; the app-layer seal + the `check:classb-read` CI guard are the whole control**. The two cross-tenant findings in this incident (`candidate_job_intelligence` by-candidate, `hiring_manager_shares`) are Class B, so prod has **no DB backstop** for them — the §2 app-layer fixes are the only seal.

Connect to the **prod** database as the table owner / superuser. Run the query below **for each Class-A table** (start with `candidates`, `applications`, `interview_sessions`, `sourced_candidates`; replace `:tbl`):

```
-- 3a. Is RLS enabled and forced on the table?
SELECT relname, relrowsecurity, relforcerowsecurity
FROM pg_class
WHERE relname = :tbl;

-- 3b. Does the tenant-isolation POLICY actually exist (and what is its expr)?
SELECT polname, polcmd,
       pg_get_expr(polqual, polrelid) AS using_expr,
       pg_get_expr(polwithcheck, polrelid) AS check_expr
FROM pg_policy
WHERE polrelid = (:tbl)::regclass;
```

- **SAFE:** 3a returns `relrowsecurity = t` **and** `relforcerowsecurity = t`, **and** 3b returns at least one policy (expected `tenant_isolation`) whose expression is `current_setting('app.is_platform_admin', true) = 'true' OR tenant_id = current_setting('app.current_tenant_id', true)`. → Tenant isolation on that Class-A table is **DB-enforced** in prod.
- **ESCALATE (this is the drizzle-push failure mode):** 3b returns **zero policies** on a Class-A table. Isolation is then **NOT DB-enforced**; the exact manifestation depends on 3a:
 - `relrowsecurity = t` (RLS enabled) + **zero policies** → **deny-all** at the DB layer — the app reads no rows for that table; or
 - `relrowsecurity = f` (the dev `(f, t)` residue: only `FORCE` is set, which is inert without `ENABLE`, so RLS is not actually on) → rows are **unfiltered** and tenant isolation is **app-code-only**. Either way this is the residue state the review flagged as **[UNVERIFIED]**; if seen, treat that table's prod isolation as app-code-only and remediate by applying the `.sql` policy migration (`0000_rls_pilot.sql` / `0001_rls_extension.sql`), **not** drizzle-kit push.
- **Class B is expected to show `(f, f)` and zero policies** — that is the designed state, not an escalation; the control there is app-code + CI guard, with no DB layer to verify.

Item 4 — Confirm the privacy-seal fixes are actually **PUBLISHED** to prod (not just merged in dev)

The fixes being on `main` in dev does not mean they are serving prod traffic.

4a. Compare the deployed build to the fix commit. In the deployment console, read the **currently-serving** deployment's source commit / build SHA and confirm it is **at or after 2026-07-07 00:42 UTC** ("Privacy incident Phase 3").

- **SAFE:** deployed commit $\geq$ that commit.
- **ESCALATE:** deployed commit is older → the sealed code is **not live**; redeploy before closing the incident.

4b. Behavioural confirmation (belt and suspenders). Item 1b already proves the seal behaviourally against the live host — a barred candidate 404-ing on the by-id career routes in prod is direct evidence the sealed code path is running. Treat 4a (SHA) and 1b (behaviour) as needing to **agree**.

Note: there is currently **no application version/build-SHA HTTP endpoint** to curl for this (health endpoints `GET /health`, `GET /healthz`, `GET /healthz/live` exist but report liveness/DB-ping, not the build SHA). Until one is added, Item 4a depends on the deploy console. Adding a

`/version` endpoint that returns the build SHA is a recommended follow-up so this check can be automated.

Sign-off

Item	Result (SAFE / ESCALATE)	Evidence captured	Checked by	Date
1 — endpoints live + sealed				
1c — no barred candidate in rec-push output				
2 — access-log review (+ logging_since recorded)				
3 — Class-A RLS DB-enforced in prod (Class-B is app-code-only by design)				
4 — sealed code actually published to prod				

Incident may be closed only when Items 1-4 are **SAFE** (or carry a tracked, accepted exception) and this sheet is signed.